

Avanade Development Guidelines for Dynamics 365 for Operations



This document provides the Development Guidelines for Dynamics 365 for Finance and Operations development work.

This document contains confidential and proprietary information of Avanade and may be protected by patents, trademarks, copyrights, trade secrets, and/or other relevant state, federal, and foreign laws. Its receipt or possession does not convey any rights to reproduce, disclose its contents, or to manufacture, use or sell anything contained herein. Forwarding, reproducing, disclosing, or using without specific written authorization of Avanade is strictly forbidden.

The Avanade name and logo are registered trademarks in the US and other countries. Other brand and product names are trademarks of their respective owners.

Table of Contents

1. Document history.....	2
2. Disclaimer.....	3
3. Introduction	5
3.1. Purpose of document.....	5
3.2. Developer prerequisites	5
3.3. Project specific guidelines.....	5
4. Acronyms.....	6
5. General development environment settings.....	6
5.1. Visual Studio	6
5.2. Best practice parameters.....	9
5.3. Solving best practice errors and warnings.....	10
6. Avanade implementation of best practices.....	12
6.1. Controllable customizations	12
6.2. Security development.....	13
6.2.1. Role based security	13
6.2.2. Record level security.....	14
6.3. Storing of sensitive information.....	14
6.4. Labels and label files.....	14
6.5. Solutions and projects.....	15
6.6. Naming conventions.....	16
6.6.1. Model naming	16
6.6.2. New objects	17
6.6.3. General naming guidelines.....	18
6.7. Coding conventions.....	21
6.7.1. Development language.....	21
6.7.2. Method parameters	21
6.7.3. Method variables.....	21
6.7.4. Proper indentation of code.....	22
6.7.5. Layout conventions.....	22
6.7.6. Code commenting conventions	23



6.7.7.	Error handling	23
6.8.	Coding patterns.....	24
6.8.1.	Frameworks.....	24
6.8.2.	Using var keyword.....	24
6.8.3.	Using statement	25
6.8.4.	Using const keyword	25
6.8.5.	Using macros	26
6.8.6.	Extensions	27
6.9.	Base enums.....	28
6.10.	Extended data types	28
6.11.	Tables.....	29
6.11.1.	Naming	29
6.11.2.	Properties	29
6.11.3.	Fields.....	30
6.11.4.	Indexes.....	31
6.11.5.	Table relations	31
6.11.6.	Temporary tables – InMemory vs. TempDb	31
6.11.7.	Data entities.....	32
6.12.	Forms	32
7.	Code review	32
8.	Suggested reading.....	32
9.	References.....	33

1. Document history

<i>Date</i>	<i>Revision</i>	<i>Author</i>	<i>Comments</i>
10.08.2016	1.0.0.0	Mohit Rajvanshy	Initial draft
28.10.2016	1.0.0.0	Marc Nelissen	Draft review



16.11.2016	1.0.0.1	Mohit Rajvanshy	Review comments implemented, addition of best practices
27.12.2016	1.0.0.1	Tarun Singh, Christian Mayr	Document Review
09.01.2017	1.0.0.2	Mohit Rajvanshy	Document update
13.01.2017	1.0.0.2	Marc Nelissen	Document review
17.01.2017	1.0.0.3	Mohit Rajvanshy	Review comments implemented, title changed
25.01.2017	1.0.0.4	Mike Urban	Document Review and update
05.07.2018	1.0.0.5	Santosh Yadav	Updated with naming details
06.07.2018	1.0.0.5	Joseph Enderlein	Updated
08.08.2018	1.0.0.5	Santosh Yadav	Update few naming and changed Avanade logo
09.10.2018	1.0.0.6	Thomas Kristiansen	General review and additions
25.10.2018	1.0.0.7	Thomas Kristiansen	Requirement to extend data entities
25.10.2018	1.0.0.7	Santosh Yadav	Updated link for learning and knowledge base and reference to Avanade Label Editor
01.11.2018	1.0.0.8	Santosh Yadav	Making doc customer specific too
22.11.2018	1.0.0.9	Alexander Zazworka	Formatting
23.12.2019	1.0.1.0	Thomas Kristiansen	General review and additions
30.12.2019	1.0.1.1	Santosh Yadav	Update against review comments
08.01.2019	1.0.1.2	Thomas Kristiansen	Client references removed Requirements for developers, project specific guidelines, password handling
18.02.2020	1.0.13	Santosh Yadav	Updated comments and added reading materials.

2. Disclaimer

THE INFORMATION IN THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE AND IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND. THE ENTIRE RISK ARISING OUT OF THE USE OF THIS INFORMATION REMAINS THE



RESPONSIBILITY OF THE READER. AVANADE HAS MADE EVERY EFFORT TO MAKE THIS INFORMATION ACCURATE AND BASED ON BEST PRACTICES AND GUIDELINES. HOWEVER, IN NO EVENT SHALL AVANADE BE LIABLE FOR ANY DIRECT, CONSEQUENTIAL, INCIDENTAL, SPECIAL, PUNITIVE OR OTHER DAMAGES WHATSOEVER (INCLUDING WITHOUT LIMITATION, DAMAGES FOR LOSS OF BUSINESS PROFITS, BUSINESS INTERRUPTION OR LOSS OF BUSINESS INFORMATION), EVEN IF AVANADE HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.



3. Introduction

3.1. Purpose of document

The purpose of this document is to provide Avanade resources guidelines for development in Dynamics 365 for Finance and Operations (D365FO). Everyone developing solutions for customers at Avanade should follow these guidelines. This includes Avanade employees as well as subcontractors who carry out development on behalf of Avanade. These Avanade development guidelines are intended to be applied on top of the D365FO development standards and best practices, you can find them at [Avanade Asset Marketplace](#), please search for "Avanade Development Guidelines for Dynamics 365 for Operations".

3.2. Developer prerequisites

Any person contributing to planning, specification, implementation, test and deployment of customizations of the D365FO solution must be properly trained and qualified. This includes:

- General programming skills and Microsoft tools (Visual Studio) in particular
- Programming languages X++, C#, SQL
- Relevant technologies like .NET and Azure
- D365FO developer certification ([Dynamics 365: Finance and Operations App Developer Associate](#))
- Member of knowledge sharing forums like Avanade [Communities of Practice](#) and Yammer groups
- Solid understanding of the functionality of D365FO and of the Microsoft technology stack
- Understanding of the customer's business in general and the business processes related to the D365 system
- Understanding of the importance of quality work, including the development guidelines

3.3. Project specific guidelines

In addition to these general guidelines it is necessary to define guidelines for the particular project or customer situation. These guidelines specify very precisely the development and deployment requirements. They must be defined before the start of the development work.

Typically, the guidelines and information specific for the project will be documented in the Wiki pages of development project in DevOps.

Mandatory	Optional
• Naming prefix and other naming conventions • Model name • Label file name • Label languages	• Installed ISV solutions with model names • Additional coding requirements • Test and deployment process • System architecture • Key persons



- Source control setup (DevOps) incl. VSTS project, branch, workspace setup, check-in requirements, use of shelvesets
- Documentation requirements

4. Acronyms

During this document, we will use following acronyms.

Acronym	Abbreviation
D365FO	Dynamics 365 for Finance and Operations
AOT	Application Object Tree
Customization code	A unique reference to a business requirement or incident that will be used throughout the whole delivery cycle
Customization prefix	A prefix for objects that will indicate that they are created by Avanade
WCF	Windows Communication Foundation
DMF	Data Management Framework
OData	Open Data Protocol
REST	Representational State Transfer
HTTP	Hyper Text Transfer Protocol
JSON	JavaScript Object Notation
EDT	Extended Data Types

5. General development environment settings

Compared to previous version of Dynamics AX, D365FO provides a different perspective and experience to developers in terms of technology, toolsets, and environments. This section provides the detail of the settings developers should utilize when setting up their development environments.

5.1. Visual Studio

Connect to the DevOps project

Configure workspace:

- Map the Metadata and Project folders of the DEV branch.
- Set the workspace to public to allow sharing among developers.



Edit Workspace PKRT81Test-1

Name:	PKRT81Test-1
Server:	pkrt.visualstudio.com\pkrt
Owner:	T Kristiansen
Computer:	PKRT81Test-1
Location:	Local
File Time:	Current
Permissions:	Public workspace

A fully public workspace can be used, checked in, and administered by any valid user.

Comment:

Working folders:

Status	Source Control Folder	Local Folder
Active	\$/test/DEV/Metadata	K:\AosService\PackagesLocalDirectory
Active	\$/test/DEV/Projects	C:\Users\Admin72da291ac5\Documents\Visual Studio 2015\Projects

A typical developer should setup their Visual Studio environment as described at the following link:

<https://docs.microsoft.com/en-us/dynamics365/unified-operations/dev-itpro/dev-tools/configure-developer-vm>

Some of the key settings are provided below. Navigate to Dynamics 365 → Options, and use the setup as indicated below.

Options

Search Options (Ctrl+E)

- Text Editor
 - General
 - File Extension
 - All Languages
 - General
 - Scroll Bars
 - Tabs
 - CodeLens
 - Basic
 - General
 - Scroll Bars
 - Tabs
 - Advanced
 - Code Style
 - C#
 - C/C++
 - CoffeeScript

Statement completion

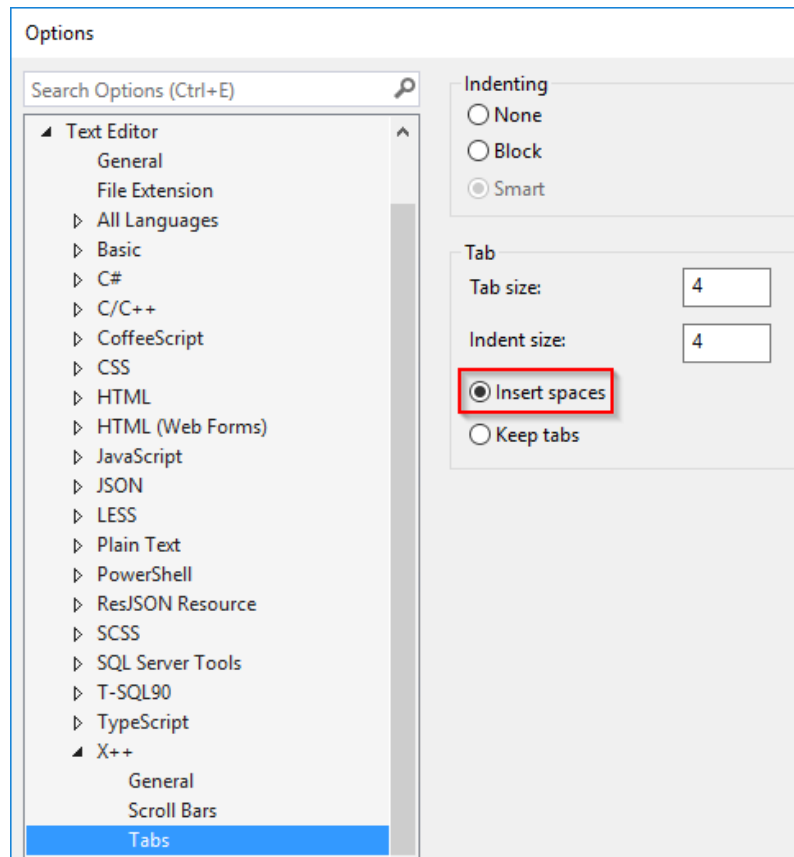
- ☒ Auto list members
- ☐ Hide advanced members
- ☒ Parameter information

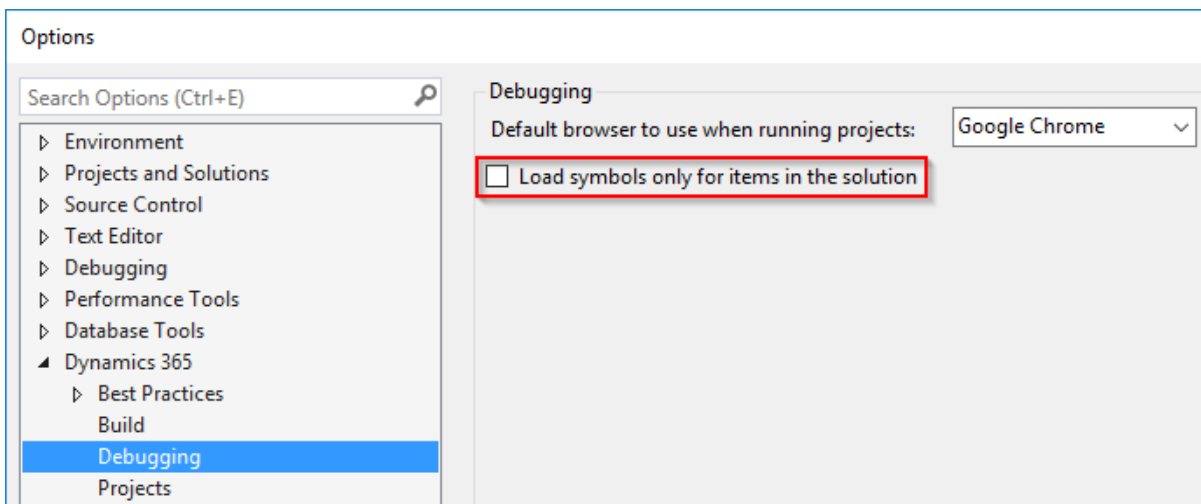
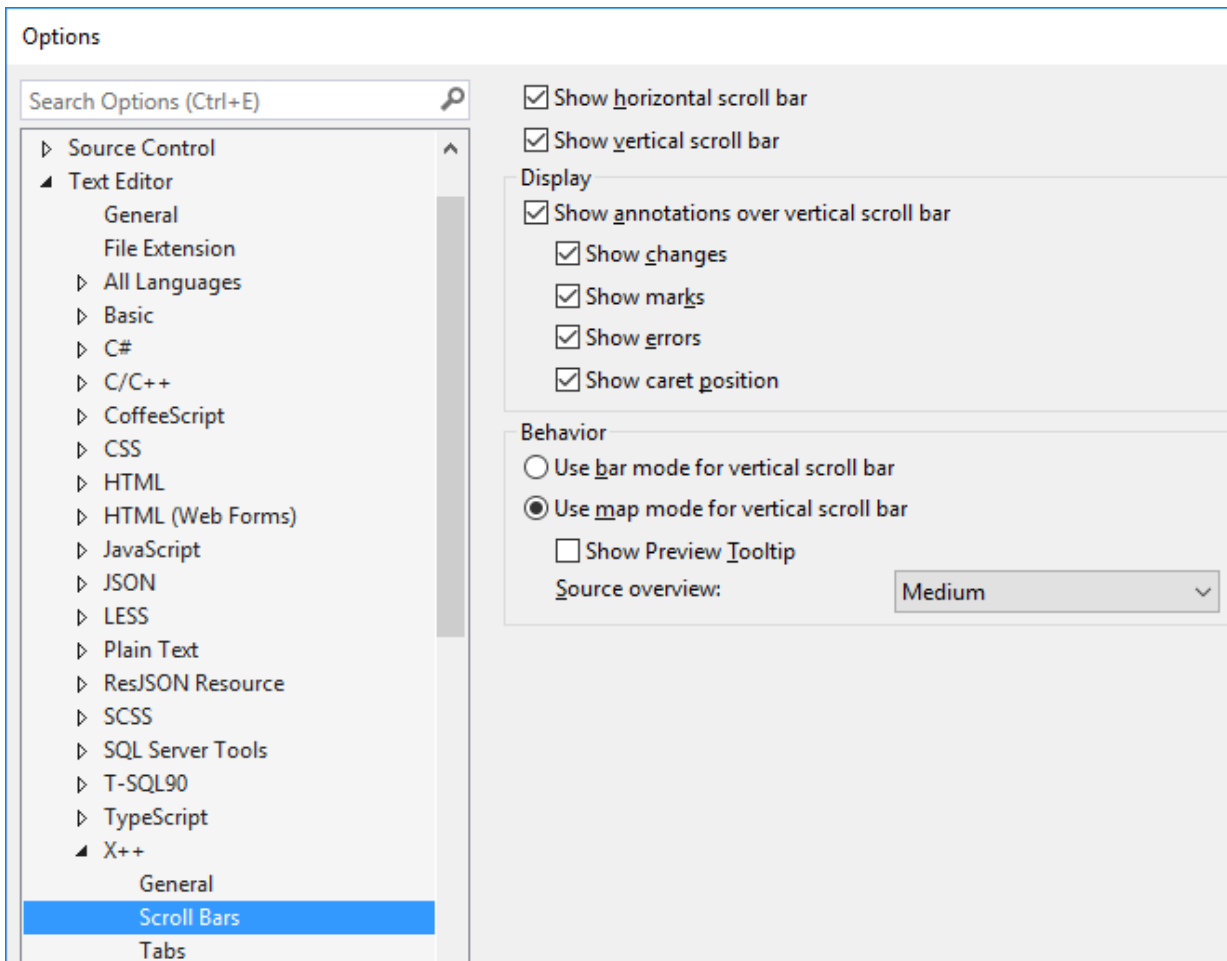
Settings

- ☐ Enable virtual space
- ☒ Word wrap
- ☒ Show visual glyphs for word wrap
- ☒ Line numbers
- ☒ Enable single-click URL navigation
- ☒ Navigation bar
- ☒ Automatic brace completion
- ☒ Apply Cut or Copy commands to blank lines when there is no selection

OK Cancel







Some valuable Visual Studio extensions for development:

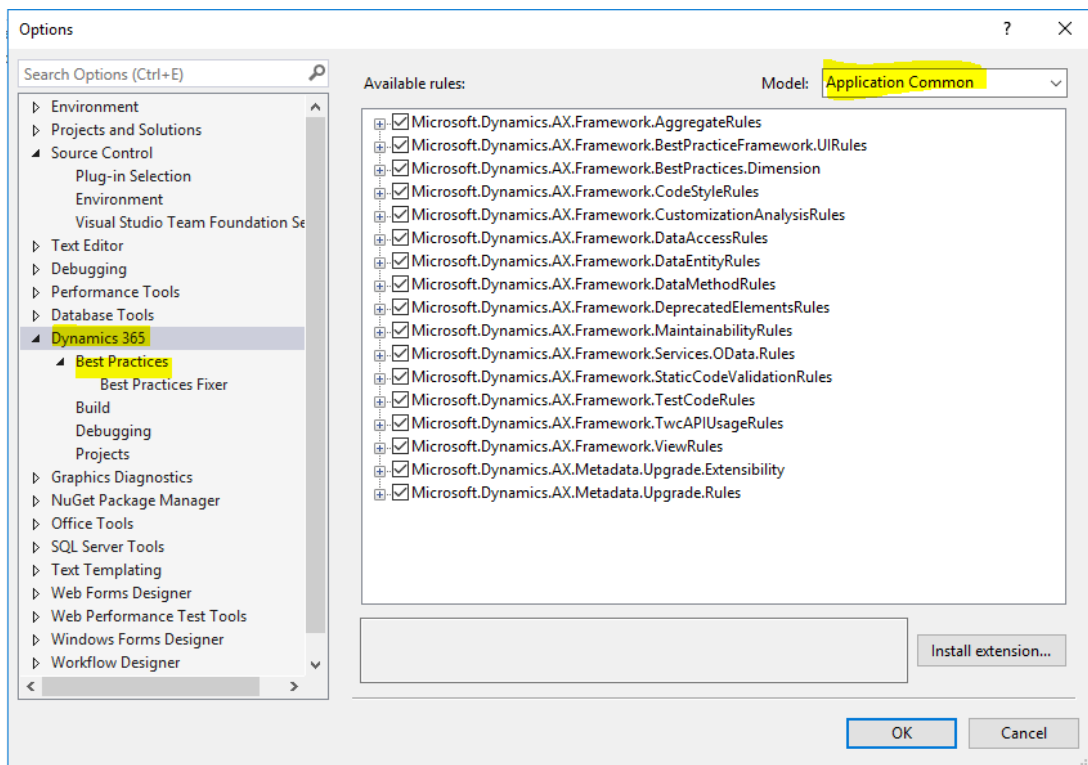
- [Avanade Label Editor](#)
- [ReAttach](#) (start the debug process with one click)

5.2. Best practice parameters

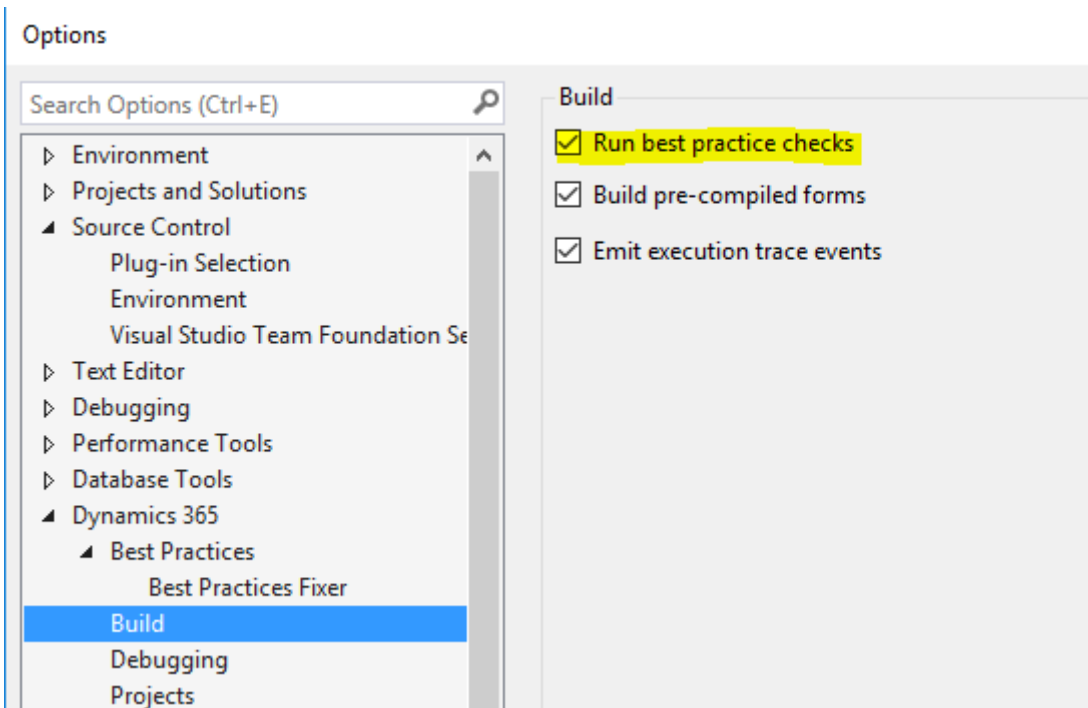


A developer should enable all best practices rules for the model(s) where they are developing their code. To enable best practice rules, navigate from the

Dynamics 365 menu → Options → Dynamics 365 → Best Practices and make sure all the rules are checked for your model(s), as shown below.



From the Build tab, you will also need to select the “Run best practice checks” option.



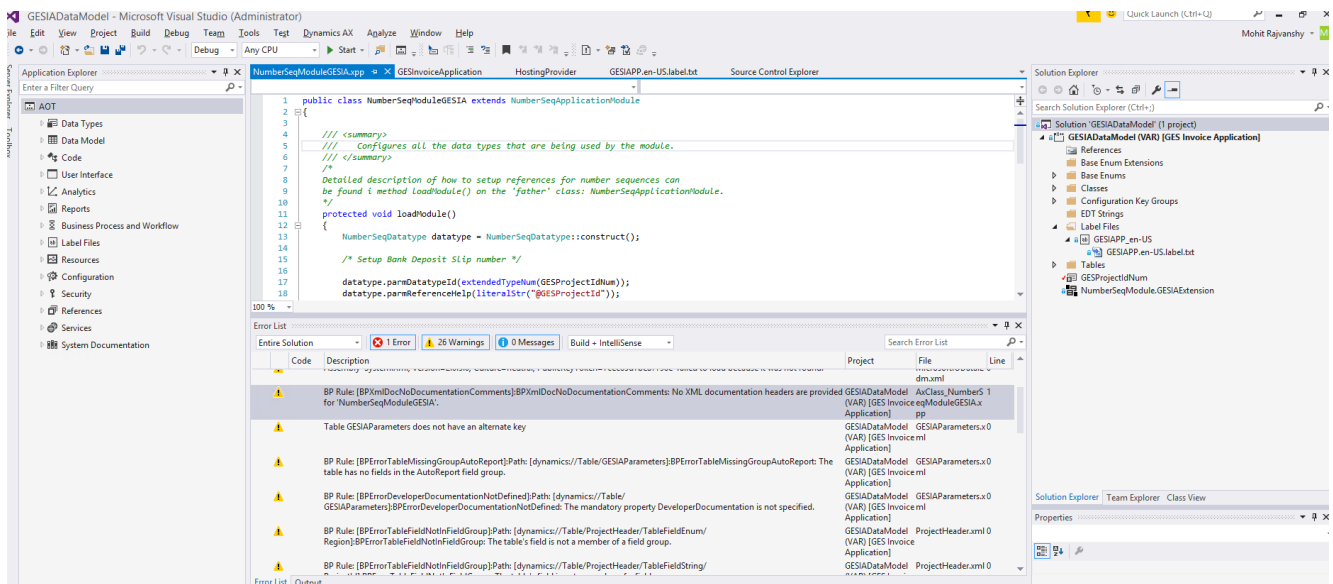
5.3. Solving best practice errors and warnings



For all development done by Avanade resources you must resolve all best practice errors and warnings before delivering final code. If best practice errors

or warnings exist in the standard solution or a third-party add-on and such objects are modified by Avanade or [Customer], then only best practice errors and warnings in Avanade/ [Customer] modifications need to be solved. Avanade solves Avanade BPs and the [Customer] solves the [Customer] BPs.

Best practice information is provided in the Compiler output window when you build a solution, please make sure you enable the Error list window. To do so, from the menu in Visual Studio go to View → Error List (Ctrl+W, E).



There are rare exceptions where BP deviations make sense: For example, the “e” parameter in event handlers:

For every deviation in code there needs to be a conclusive comment in the method:



[SuppressBPWarning('BPPParameterNotUsed', 'Parameter e required in signature'),
FormDataSourceEventHandler(formDataSourceStr(CustPaymentEntry, LedgerJournalTrans), FormDataSourceEventType::Created),
FormDataSourceEventArgs e)]

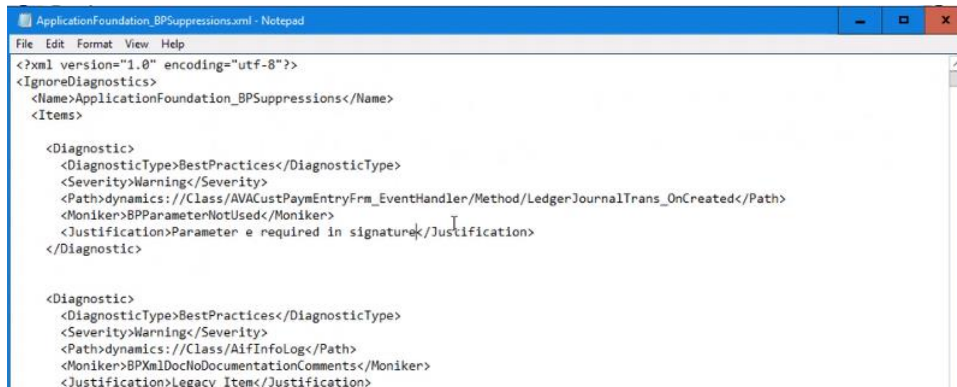
For non-code deviations the conclusive comment must be placed in a best practice exceptions file.

This file must be created per model and placed in the package folder:



[\\PackagesLocalDirectory\<ModelName>\<ModelName>\AxIgnoreDiagnosticList\<ModelName> BPSuppressions.xml](#)

In the Directory folder there is an example (Directory_BPSuppressions.xml).



```

<?xml version="1.0" encoding="utf-8"?>
<IgnoreDiagnostics>
  <Name>ApplicationFoundation_BPSuppressions</Name>
  <Items>

    <Diagnostic>
      <DiagnosticType>BestPractices</DiagnosticType>
      <Severity>Warning</Severity>
      <Path>dynamics://Class/AVACustPaymEntryFrm_EventHandler/Method/LedgerJournalTrans_OnCreated</Path>
      <Moniker>BPPParameterNotUsed</Moniker>
      <Justification>Parameter e required in signature</Justification>
    </Diagnostic>

    <Diagnostic>
      <DiagnosticType>BestPractices</DiagnosticType>
      <Severity>Warning</Severity>
      <Path>dynamics://Class/AIfInfoLog</Path>
      <Moniker>BPXmIDocNoDocumentationComments</Moniker>
      <Justification>Legacy Item</Justification>
    </Diagnostic>
  </Items>
</IgnoreDiagnostics>
  
```

6. Avanade implementation of best practices

6.1. Controllable customizations

NOTE: This section may not apply to all project but will apply if there are multiple instances or if Avanade is building customizations that will become supported assets. It is still good practice to include this though if a customer ever decides to turn on/off features.

It is recommended to use a configuration key or a parameter to build modular solutions that can easily be ported to other customer environments. The choice of utilizing a configuration key or a parameter depends on the type of customization you are developing. There is a clear distinction between these two options:

- Configuration Key
 - A configuration key specifies if the new developed functionality will be available to the entire system. So, whenever you have developed a customization that should be made activated or deactivated on a system-wide level, a configuration key should be utilized.
- Parameter
 - A parameter specifies the way in which the new developed functionality is modeled and allows for different settings for different companies within the system. So, whenever you have developed a customization should be activated or deactivated on a company-level, a parameter should be utilized. If you create a new module, add a new parameter table. If you extend an existing module, extend the related parameter table. Also, follow the way standard parameter tables are defined.

The most common usage of configuration keys is implemented on data dictionary objects. Tables, for instance, have a configuration key property that will ensure that they exist in the underlying database when the configuration key is enabled.



Functionality in code can also be linked to the state of a configuration key. Refer to the example below to check if a certain piece of code should be executed only when a configuration key is enabled:

```
// Check if configuration key AVANewModule is enabled if
(isConfigurationkeyEnabled(configurationkeynum(AVANewModule)))
{
    // Execute code
}
```

Note - This call is expensive if you do it every time before use, to avoid this it should be executed in an init method or initialization of a form and then put into a local variable where it can be checked against without having to make the call to check the key again.

To further increase performance, it is better to cache this variable as follows.

```
public class AVAGESInvoiceHandler
{
    boolean isConfigEnabled;

    public void initialize()
    {
        // Check if Config key is enabled and then assign to local variable
        if (isConfigurationkeyEnabled(configurationkeynum(AVANewModule)))
        {
            isConfigEnabled = true;
        }
    }

    private void createInvoiceReport()
    {
        if(isConfigEnabled)
        {
            // Perform operation
        }
    }
}
```

Every newly created table, view and menu item must have a configuration key, which can be one from the standard solution.

For further details on which configuration key to use for what purpose, please consult with your technical architect or development lead if no details are included in the technical design.

6.2. Security development

6.2.1. Role based security



Each menu item should be associated with two privileges (maintain and view) which internally will be part of duties and duties will add to a specific role.

Depending upon the scenario duties could be part of any existing role or if needed a new role could be introduced.

Setting up and applying security roles is part of the customers' responsibilities. As a developer we only have to make sure that all new menu items are bound to the security framework and that the security elements are documented in the release protocol. Normally a new menu item requires one or two new privileges (view and maintain access). If new duties and roles are required or if the privileges can be bound to existing ones depends on the specific requirements.

6.2.2. Record level security

If required use the extensible data security (XDS) framework to secure data with query-based restrictions. A good explanation of how to use XDS policies is found in this blog:

<https://alexdmeyer.com/2019/02/20/extensible-data-security-xds-framework-in-d365fo/>

Beware that applying an XDS policy has a performance impact, so consider carefully the fields used for filtering and the relation between the constrained and primary tables.

6.3. Storing of sensitive information

In general, it is not recommended to store connection information and passwords to other systems in the D365FO (AXDB) database, like in a parameter table. Shortcomings:

- If not encrypted, a security threat, as an unauthorized person could access the information.
- If encrypted, the data is lost when copying the database.
- The data must be updated when copying the database from one environment to another to align with the related systems (risk of updating production data from test environment when not done properly).

Encrypting the data of a field can be obtained by implementing an edit method with SysSQLPassword data type using the editEncryptedField method of the Global class.

The better way to store sensitive information is to integrate Azure Key Vault. This will allow for a better password management enterprise-wide, e.g. different passwords for production and test systems. The connection information to Key Vault must be stored only once. It is also possible to restrict access to access only from certain URLs thus avoiding the risk of accidentally production data from the test environment.

For instructions how to use the Key Vault see this blog:

<https://www.agermark.com/2018/09/how-to-setup-azure-key-vault-with.html>

6.4. Labels and label files

Please implement the following best practices while creating labels and label files -

1. All UI texts that are created in customizations must be created by utilizing a label in a specific label file.
2. Create a label file with the same name as the associated model.



3. Use labels when programming for all external strings.

4. There should be at least one specific label file per model.
5. Before you create a new label, always make sure to do a get latest on the label file you are editing.
6. Labels should be given a meaningful name. The label must be an abbreviation of the label text and always in English. Do not use any prefix. Please ensure that labels contain the appropriate translation of activity or item.
7. All defined languages in the project should filled. If you don't know the translation, choose the English words.
8. Use the description field to reference the task.

Some examples of UI texts:

- The caption of a form
- The label for a field
- A message displayed to the user

Label ID	Label	Description
ProductOwner	Product owner	259 - MDM02
ProductOwnerHelpText	Product owner, initialized with the origin company which created the product	259 - MDM02

6.5. Solutions and projects

A solution is a structure for organizing projects in Visual Studio. A Visual Studio solution could contain several projects. The solution maintains the state information for projects in .sln (text-based, shared) and in (binary, user-specific solution options) files.

The name of the Solution must be prefixed with **AVA** or a [Customer] agreed upon prefix and must contain the reference to the project implementation VSTS Task.

Example: **AVA_004711_LO01_SalesReports**

The Dynamics 365 project type is part of the Visual Studio extension Dynamics 365 Unified Operations platform. This project type resembles other projects in Visual Studio.

Please note: A project can contain elements from only one model. If you must work with elements from different models, you must use multiple projects in your Visual Studio solution.

The name of the project must be prefixed with **AVA** (or other customer agreed upon prefix) and must contain the reference to the use case (or business requirement ID) or incident (issue ID). In fact, this is the identification of the customization that will be used throughout the whole delivery cycle and enables traceability from business requirement or incident to functional analysis, technical analysis and even comments in the code. This will allow a developer to trace a piece of code all the way back to the reason why it was implemented.



Add the solution and project to source control.

Create a new project only if a new requirement with a new identification (and new DevOps task) exists. Otherwise reuse the existing project.

6.6. Naming conventions

Naming conventions contribute to consistency and making the application easier to understand. A typical customization consists of extended objects (as all models locked with spring release). To distinguish customizations of standard or third-party objects from their standard implementation and new objects from base objects a customization-prefix must be used.

Unless specified otherwise by the technical architect or development lead of your project this customization prefix has to be **AVA** (ava for methods).

6.6.1. Model naming

Unless specified the model should be named as **AVA[ModuleName][FeatureName]** or **(Customer specific)**. Specific values should be provided for model description and model display name.

We should have one package against one development but inside that we can have multiple projects to cater features.

The screenshot shows a 'Create model' dialog box. On the left is a 'Steps' sidebar with four items: 'Add parameters' (highlighted in blue), 'Select package', 'Select referenced packages', and 'Summary'. The main panel is titled 'Add parameters' and contains the following fields:

- Model name:** A text box containing 'AVA<customername>'.
- Model publisher:** A text box containing 'Avanade Inc.'.
- Layer:** A dropdown menu with 'isv' selected.
- Version:** A text box containing '1.0.0.0'.
- Model description:** A large empty text area.
- Model display name:** A text box containing 'AVXGESInvoice'.

At the bottom of the dialog are three buttons: 'Back', 'Next', and 'Cancel'.

Select the "Create new package" option while creating a model. All development must to be done using extension framework only.

In next page please select base packages 'Application Foundation', 'Application Foundation' and Application Suite'. Few more packages (Ledger, Directory, Dimension, Calendar & Currency etc.) also make sense to choose as it contains other most frequently needed objects for extensions.

Create the new model and new package in the lowest layer, you could add a new model in upper layers in the same package. As an example:

- Create model A in VAR layer and create a new package (VAR package). Create model B in CUS layer and add it to existing package (VAR package).
- This package will have both the models from different layers, hence while creating a deployment package you will have both the models included.

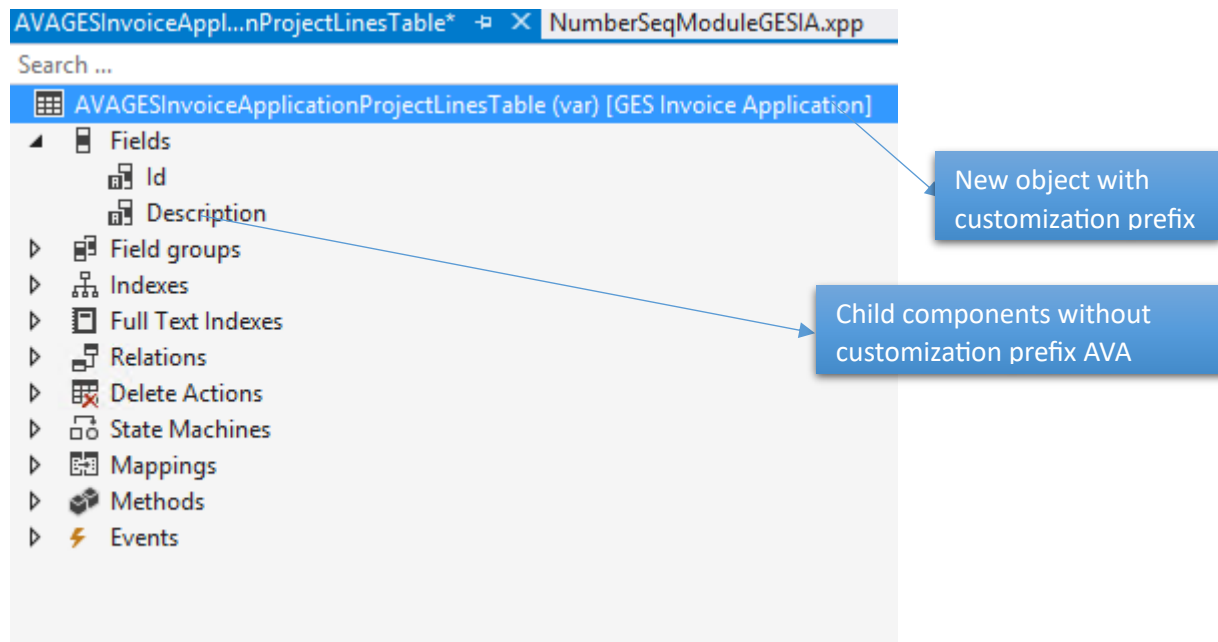
Creating separate packages for the VAR layered model and the CUS layered model and then combining both packages is not possible.

6.6.2. New objects

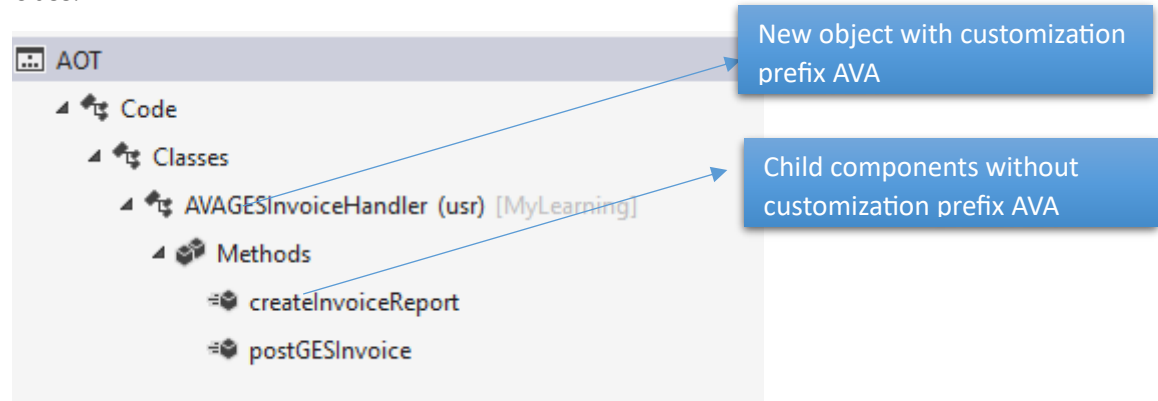
The name of all new objects that are created by Avanade must have the customization prefix, but their child components must not.

The following is an example of a new table:





An example of a new class:



Note: An exception are extension classes where the method names must be prefixed with ava in order to differentiate them from the main class/table/form method names.

6.6.3. General naming guidelines

- Wherever possible, application object names should be constructed hierarchically from three basic components:
 - the business area name,
 - the business area description,
 - and the action performed for classes or the type of contents for tables.

For ex: AVAStoreReplenishmentParameters is a parameter table for store replenishment functionality.

- All names should be in U.S. English, and the default rule is to use logical and descriptive names if no other specialized rules apply.



3. Names in the Application Explorer and X++ code must correspond to the names in the U.S. English user interface.
4. Names should always be spelled correctly and used consistently.
5. Each path in the AOT must be unique, nodes cannot have identical names.
6. Do not begin a name with AAA or copyOf. For ex: **AVACopyOfStoreReplenishmentParameters** is not allowed.
7. Do not begin a name with DEL underscore unless it's a table, extended data type, or enum that is needed for data upgrade purposes.
8. It's also important to prefix certain elements that relate to Customers with Cust, Production elements with "Prod", Vendor elements with Vend, and Inventory elements with Invent. This will create continuity with the existing system elements. For ex: **AVACustInvoiceMap**



9. **Naming of Event handler classes:** **AVA**<Objectname> <Type abbreviation> **_EventHandler**

Object type	Abb.	Example
Table	Dbt	AVASalesTableDbt_EventHandler
Form	Frm	AVASalesTableFrm_EventHandler
Class	Cls	
Map	Map	
Data entity	DE	

10. **Naming of MetaData Extension Objects:** Object.AVAExtension

Extensions of tables, queries, views, EDTs, enums and forms.

Example: SalesTable.AVAExtension

Object type	Abb.	Example
Table		SalesTable.AVAExtension
Form		SalesTable.AVAExtension
Class		

11. **Naming of Extensions** (Chain of command and adding methods to existing classes) such as Object_Extension using the ExtensionOf attribute

Add Prefix and Type abbreviation: **AVA**<ObjectName> <Type abbreviation> **_Extension**

Object type	Abb.	Example
Table	Dbt	AVASalesTableDbt_Extension
Form	Frm	AVASalesTableFrm_Extension
Class	Cls	AVASalesTableTypeCls_Extension

Include the parent form name followed by **Frm** (and the data source name followed by **DS** for datasource fields) for nested types such as FormDataSource, FormDataField or FormControl.

Nested type	Abb.	Example
FormDataSource	DS	AVASalesTableFrm_SalesTableDS_Extension
FormDataField	Fld	AVASalesTableFrm_SalesTableDS_SalesIdFld_Extension
FormControl	Ctrl	AVASalesTableFrm_MCRCustomerServiceCtrl_Extension

12. See also the naming guidelines from Microsoft:

<https://docs.microsoft.com/en-us/dynamics365/fin-ops-core/dev-itpro/extensibility/namingguidelines>

13. Name internal variables in camelCase.



6.7. Coding conventions

6.7.1. Development language

At all times, all development must be carried out in the US English (this is a Microsoft best practice) to ensure proper readability for every future developer that might work on that development.

This applies to:

- Names of new objects and child components
- Variable and parameter names
- Header and inline comments

6.7.2. Method parameters

Method parameter names must be prefixed with an underscore followed by a lowercase character the rest of the method parameter name must be in camelCase. (e.g. _projName).

If a method has multiple input parameters, they must be placed one under the other and at the same level of indentation (the types as well as the parameters) to aid in the readability of the method.

D365FO uses the .NET runtime, aka CLR. This has some implications for method signatures. In the CLR a method's signature includes casing, parameters, and return type. In AX2012 a method's signature was simply the method's name – case insensitive. However, in D365FO, it is case sensitive, hence precaution needs to be taken while changing the casing.

6.7.3. Method variables

Method variable names must start with a lowercase character. The rest of the method variable name must be in camelCase. No underscores are allowed in method variable names.

If multiple method variables are present in a method, they must be placed one under the other and at the same level of indentation.

- Bad:

TableBufferA	tableBufferA;	
TableBufferABC	tableBufferABC;	Not properly indented
AvaClassA	avaObjectA;	

- Good:

TableBufferA	tableBufferA;	
TableBufferABC	tableBufferABC;	Good readability
AVAclassA	avaObjectA;	

D365FO supports declaring variables close to where they are used, meaning inline variables. Variables are scoped by the block in which they are declared, and variable names can be reused in another scope.



6.7.4. Proper indentation of code

Conditional execution of code must always be indented correctly and placed on the line after the condition

Bad: Not indented properly

```
if (myCondition == true)
{
    // Do some coding          Not indented properly
}
```

Bad: Code on same line as condition

```
if (myCondition == true) // Do some coding
```

Good:

```
if (myCondition == true)
{
    // Do some coding    Indented properly and on line after condition
}
```

6.7.5. Layout conventions

Good layout uses formatting to emphasize the structure of your code and to make the code easier to read. Microsoft examples and samples conform to the following conventions:

- Use the default Code Editor settings (smart indenting, four-character indents, tabs saved as spaces). For more information, see [Options, Text Editor, C#, Formatting.](#)
- Write only one statement per line.
- Write only one declaration per line.
- If continuation lines are not indented automatically, indent them one tab stop (four spaces).
- Add one blank line between method definitions and property definitions.
- Use parentheses to make clauses in an expression apparent, as shown in the following code.

```
void MyMethod()
{
    for (int i = 0; i < 10; i++)
    {
        info(strfmt("i is %1", i));
    }
}
```

- In Visual Studio the methods start with one tab. Everything within the method starts at two tabs.
- The starting parenthesis on method declarations and calls should be the character just after the method name (no space).
- If there are one or two parameters, the parameters can be listed on the same line. If there are



more than two parameters, move each parameter onto a new line, and indent by 4 spaces.

- Use a single blank line to separate entities.
- Do not use parentheses around the case constants.
- Do not use parentheses around where expressions.
- Add one space between if, switch, for, while and the expressions starting parentheses.
- Use braces around all code blocks, except for around case clauses in a switch statement.
- Use braces even if there is only one statement in the block.

6.7.6. Code commenting conventions

Code comments are required for two purposes:

- Class and method descriptions in the header, formatted in the proper xml syntax:
 - Description of class/method
 - Input parameters
 - Return values
 - Each line must start with ///.
- Explanation of the code:
 - Place the comment on a separate line, not at the end of a line of code.
 - Begin comment text with an uppercase letter.
 - End comment text with a period.
 - Insert one space between the comment delimiter (//) and the comment text, as shown in the following example.

```
void MyMethod()
{
    // Loop through counter.
    for (int i = 0; i < 10; i++)
    {
        info(strfmt("i is %1", i));
    }
}
```

6.7.7. Error handling

- All errors must be caught in code.
- An error message must be displayed. The message should include a maximum of details about the problem and the data so that the cause can easily be identified.
- The transactional integrity must be enforced (ttsbegin/ttscommit/ttsabort). No halfway posted data that cannot be corrected. No need for ttsabort if try/catch



implemented. Which means any database operation must needs to perform inside `ttsbegin` & `ttscommit` blocks.

- If possible, only the erroneous data must be skipped. The rest of the data should be processed normally.
- A path to process the erroneous data must be available (edit and resubmit).

X++ now has support for the *finally* statement. It has the same semantics as in C#. It is recommended to write finally statement with a try catch block. It will clean up any usage of non-managed resources. The *finally* block is guaranteed to be executed, regardless of exceptions or transactions.

```
try
{
}
catch
{
}
finally
{
}
```

6.8. Coding patterns

6.8.1. Frameworks

Apply the provided framework classes as much as possible, especially the SysOperation framework instead creating classes extending RunBase.

6.8.2. Using var keyword

This is the new language feature. Just like C# you can now use the `var` keyword when declaring variables. It has the exact same semantics, and any guideline you will find on how to use it in C# will also apply in X++.

However, we are recommending the limited usage of `var` keyword preferably in following two scenarios.

1. When the type is obvious.

In X++ there is a lot of repetition. The `var` keyword can help avoid some of this clutter while making the code easier to read, without any information gets lost to the reader.

For example:

```
MyClass myClass = new MyClass();
```

In D365FO we could write:

```
var myClass = new MyClass();
```

2. When the type is irrelevant.

Sometimes we don't care about the type and nor does the reader.



For example:

```
ListEnumerator enumerator = myCollection.GetEnumerator();
```

In D365FO we would write:

```
var enumerator = myCollection.GetEnumerator();
```

There are a number of X++ specific reasons to not use var (i.e. doesn't apply to C#)

1. Limited support Visual Studio. In the X++ editor in Visual Studio, you can use F12 to go to the declaration of a type. However, the X++ editor is still not as smart as the X++ compiler. At the time of this writing, we will lose this drill through capability when not explicitly specifying the type of the variable. The same applies for IntelliSense – when using the var keyword, IntelliSense is not available (as of now).

2. Higher risk of runtime errors.

Just like the C# compiler, the X++ compiler will determine the type from the assignment to the variable and enforce all the same rules and restrictions. However, the X++ compiler doesn't enforce any validation of method invocations on object or common. So, if the X++ compiler determines the type to be an object or common, then you lost your safety net. It is not always obvious.

Consider this code:

```
CustTable custTable = Xml2Record(xml);
custTable.someInvalidMethod(); // Compile error
```

Using var, the code becomes:

```
var custTable = Xml2Record(xml);
custTable.someInvalidMethod(); // No compile error, as Xml2Record() returns a
common
```

6.8.3. Using statement

The "using" keyword increases the readability of the code segment. For example, if you want to implement type System.IDisposable interface you could write as follows:

```
using(var myObject = new MyObject())
{
    myObject.someMethod();
}
```

This is short hand for:

```
var myObject = new MyObject();
try
{
    myObject.someMethod();
}
finally
{
    myObject.Dispose();
}
```

6.8.4. Using const keyword

D365FO supports the "const" keyword which is semantically identical to [const in](#)



We recommend that you always define members on a class and variables in a method that can only be initialized during the declaration using `const` keyword. This will replace the macro usage. Usage of `const` also increases the compiler performance comparing to macros.

```
void constsUsage()
{
    #define.oneNumber("AX 2012");
    const str OneNumber = "Dynamics 365 for Operations";
    info(#oneNumber);
    info(OneNumber);
}
```

Use the `const` keyword with public and static class members to create classes with constants in order to replace macro libraries

```
static class TimeConstants
{
    static const public int DaysPerWeek = 7;
    static const public int HoursPerDay = 24;
    static const public int HoursPerWeek = TimeConstants::DaysPerWeek * TimeConstants::HoursPerDay;
}
```

Please note: if the value of a variable cannot be changed after the first assignment then `readonly` should be used. `Readonly` allows you to define members on a class that can only be initialized in the declaration and in the constructor.

```
class MyClass
{
    readonly str identifier = "XYZ";
    readonly str identifier2;
    public void new(str _identifier)
    {
        identifier2 = _identifier;
    }
    public void foo()
    {
        // The field 'identifier2' is read only. A value cannot be assigned to it.
        // identifier2 = "ABC";
    }
}
```

6.8.5. Using macros

You should limit the usage of macros and possibly use constants as much as possible. Compared to macros, constants have the following benefits:

- You can add a documentation comment to a constant, not to the value of a macro. Ultimately, the language service will pick this up and provide good information to the user.
- A constant is known by IntelliSense.



- A constant is cross-referenced, so you can find all references of a particular constant. This is not the case for a macro.

- A constant is subject to access modifiers, either private, protected, or the public. The accessibility of macros is not rigorously defined.
- Constants variables have a scope, while macros do not.
- You can see the value of a constant or a readonly variable in the debugger.

6.8.6. Extensions

Extensions are mandatory whenever you are customizing a standard object. D365FO supports extension methods similarly to [C# programming guide](#)

Ex: Add a method to the DirPersonName table in D365FO. We need to perform this action without modifying the DirPersonName table

```
static class MyDirPersonName_Extension
{
    static public PersonName avaFullName(DirPersonName _person)
    {
        return strFmt('%1 %2 %3', _person.FirstName, _person.MiddleName, _person.LastName);
    }
}
```

Things to remember:

1. Only one Extension per Object and Model.
2. The class must be postfixed with "_extension".
3. The class must be static.
4. The extension methods must be static.
5. The type of the first parameter determines which type is extended.
6. Use the customization prefix for the method name (usually ava).
7. Always use Chain-of-Command extensions instead of event handlers if available.
8. Do not include event handler methods in extension classes (with the ExtensionOf attribute). Instead create a separate class for event handlers.

Now you can enjoy your extension method:

```
DirPersonName dirPersonName;

while select dirPersonName
{
    info(dirPersonName.avaFullName());
}
```

Note:

1. When calling extension methods, you don't provide the first parameter – that gets inferred from the instance's type.



2. If the extension method took any additional parameters – they need to be provided.
3. This doesn't break encapsulation. The extension method only has access to public fields and methods on the type.
4. To extend a method in a form, create form extensions in the application explorer.

6.9. Base enums

Always use an existing enum except when there is no matching enum.

Please consider the following best practice recommendations when deciding to include a base enum in your customization:

1. When using the base enum on mandatory field on a table, please ensure to make the first outcome with the value zero as none and set the label property to not selected.
2. If you need to set a default value for base enumeration, the value of the zero must be used.
3. Always let the constant be an enumerator (`SalesTable.SalesType == SalesType::Sales`).
4. Never use numeric constants instead of enums (`SalesTable.SalesType == 1`).
5. Never use other constants such as other enum types instead of enums.
6. Do not make Boolean tests on enum fields except where the zero or false value is clearly the one with the false outcome.
7. You should never use relational operators on enums (`</>/=`)
8. You should also never compare or assign to enums of different types.
9. Do not expect the enumerators to have a numeric value in the range of zero which is the number of enumerators minus one.
10. Consider the data related to base enumeration if you are trying to delete, modify base enum or its values.

6.10. Extended data types

Please consider following best practice recommendations when creating an extended data type:

1. You should only create subtypes when they are needed and there is not a suitable EDT available out of box. If only the label is different use the available EDT and set the label on the table field property. If you want to add a field to a table or to a dialog, do not choose a type that is too general. Instead, create the type from the beginning and reuse it everywhere else.
2. Create one EDT for each atomic entity. EDT should pass the "is a" test for ex:
subitem ID is an item ID but it is not a class name.

3. Do not directly use the system data type `RecId` or `TableId`, instead use extended data type `RefRecId` or an EDT which is derived from `RefRecId`.



4. The help text property cannot be the same as the label.
5. Set display length and style property as Auto.

6.11. Tables

Please consider following best practice recommendations when creating a table:

6.11.1. Naming

1. New table names should start from **AVA** or [Customer] abbreviation unless otherwise stated.

AVA<ModuleName> <ProcessName>

AVACustVendTrans

2. Temporary tables names should have Tmp word in the object names. This can be at the end of the name so that the table is listed next to other tables in the same module in the Application Explorer.

AVATmpCustVendTrans

6.11.2. Properties

1. Label property should be setup.
2. For tables where the FormRef property is set to display menu item, the display menu item must exist.
3. The title fields properties include title field one and title field two.
 - Title field one should be a key field for the records in the table. Use a descriptive title if the key field has information for the user.
 - Title field two should be a descriptive field for the records in the table.
 - For example, the InventItemGroup table, the title field one property is item group ID, and the title field two property is set to name. If the value of title field one and title field two are the same, an error will occur.
4. The table group property is a mandatory property set to group for tables that contain grouping and categorization information. Deleting records from a group table can sometimes result in an unstable situation. Enable confirm deletion for any group tables such as CustGroup, VendGroup. Recommendations for the property caching per table group:

Table Group	Best Practice Caching Type
Miscellaneous	None
Parameter	Entire Table



Group	Entire Table, Found, FoundAndEmpty
Main	Found or FoundAndEmpty
Transaction	NotInTTS
WorksheetHeader	NotInTTS
WorksheetLine	NotInTTS
Framework	-
Worksheet	NotInTTS
TransactionHeader	NotInTTS
TransactionLine	NotInTTS

5. The table type property is used to set if it is a regular table or a temporary table. We recommend using the setTmp method to make a non-temporary table temporary rather than creating a copy of the table and changing the property. This way, the two versions of the table do not become out of sync.

6.11.3. Fields

1. Always use extended data types or enums as fields on the table.
2. Fields that contain the same information should share the same extended data type or base enum.
3. Use the RefRecId extended datatype for fields that refer to RecId.
4. Consider the usage of memo and container fields in application tables as they take up a lot of space. In some scenarios, you can use text fields instead of memo fields. For ex: use the Addressing data type for address related fields. Use ItemFreeTxt extended data type for item description and order lines.
5. Always use field groups to organize related fields. All fields should be in at least one table group.
6. You should place at least two fields in the auto report field group for each table except for parameter tables. The fields placed in auto report field group should be the ones that the user expects to print when they first click on print from the file menu to run the auto report. We recommend putting title field 1 and title field 2 in this field group.
6. Consider setting up the field properties AllowEdit, AllowEditOnCreate, Mandatory and Visible.
 - AllowEditOnCreate: allow edit before the record is saved.
 - AllowEdit: allow edit on an existing record. Usually this is set to Yes. In this case, the field will be editable when the record is created. Set to no when the field would be set programmatically for new records.



Note: If AllowEdit is set to no and AllowEditOnCreate is set to yes, you can edit the field on a new record but not on saved records.

- The Visible property is usually set to yes, but it can be set to no for system only information that will not be shown in the user interface.

6.11.4. Indexes

1. Always maintain indexes in the development environment using the element designer on a table.
2. For each select statement and query on the table there must be an appropriate index used.
3. Always specify a primary index and clustered index.
4. Limit the number of columns to three in an index. If you need more columns to be added please consult your technical lead and discuss the requirement.

6.11.5. Table relations

1. The name of the relation should contain the name of the table it relates to.
2. A relation should be defined in the table holding the foreign key, typically the child table.
3. The Validate property on the relation must be set to "yes".
4. If you are using a configuration key for relation, the key that is specified for the field in a relation must be the same key or in the same tree of keys as the one used in the related field.
5. Navigational relationships do not enforce integrity constraints, but they're used for navigational purposes. For example, when a form is opened within another form. The validate property on these should be set to no. Defining navigation relationships make the application easier to navigate.
6. The OnDelete property value is located within the relationship properties and it dictates what should happen when a record in the related table is deleted. Since we typically set the relation on the child table, the onDelete property specifies what should occur when a related record in a parent table is deleted. This is used to maintain database consistency when a record is being deleted in a table that has related records in another table.
 1. **None:** The child table record is not deleted when the parent table record is deleted.
 2. **Cascade:** The child table record is deleted when the parent table record is deleted.
 3. **Restricted:** The parent table record can only be deleted if there is no child table record.
 4. **Cascade + Restricted:** Cascade delete even though records exist on the child table. A warning appears; if clicking No nothing occurs; if clicking Yes the cascade deletion runs.

6.11.6. Temporary tables – InMemory vs. TempDb

The choice of whether to use a TempDB or InMemory temporary table depends on the scenario. Usually, TempDB temporary tables are faster than in InMemory ones. They are maintained in the database and support joins with regular tables.



1. Considering using InMemory temporary tables when the amount of data is small and Microsoft SQL Server round trips should be avoided.
2. An InMemory table cannot be joined in a X++ SQL statement with Regular/TempDB tables. It must be the outer table in a join. It is better to use TempDB temporary tables because SQL operations are faster due to being maintained in the database.

6.11.7. Data entities

Data entities are used for data migration and integrations with other systems. To enable these integrations, it is necessary to have all data represented in data entities:

1. All tables (except temporary) must be mapped to a data entity. Create new data entities when creating new tables. We don't need to create entities for all tables unless we know the usage of those in data migration/data related stuff.
2. Add new fields to the existing data entities when adding fields to a table (standard or custom).
3. You need to have data entities created for all your master, setup and parameter tables.

6.12. Forms

A form pattern other than Custom must be applied to all forms. For more information on form guidelines see <https://docs.microsoft.com/en-us/dynamics365/fin-ops-core/dev-itpro/user-interface/general-form-guidelines>.

7. Code review

Reviewing the code is one of the essential tasks to take care in any of the development project. To keep that in place, it's also recommended to keep this information in guideline document.

Below is the attached document prepared by ATCI team. The intent of this document is to describe the code review process using visual studio (shelveset and code review flow) and narrate all the steps with clean screenshots involved in the entire flow.



Visual Studio - code review process.pdf

8. Suggested reading

We suggest reading the following material in order to better understand best practices for setting up the development environments and some additional advanced topics.

1. Setting up a development environment for D365FO.



<https://avanade.sharepoint.com/sites/Knowledge/ layouts/15/DocSetHome.aspx?id=/sites/Knowledge/Contributions/933c3007-2857-430c-9c2f-b56e0d367b09>

2. Developers guides for 'edit in excel' experience

<https://avanade.sharepoint.com/sites/Knowledge/ layouts/15/DocSetHome.aspx?id=/sites/Knowledge/Contributions/444c3bd9-3c0a-400f-9aa7-687be709ed80>

3. Continuous delivery

<https://docs.microsoft.com/en-us/dynamics365/fin-ops-core/dev-itpro/dev-tools/continuous-delivery-home-page>

4. GDPR - <https://docs.microsoft.com/en-us/dynamics365/fin-ops-core/dev-itpro/gdpr/gdpr-guide>

5. Upgrades, updates and hotfix - <https://docs.microsoft.com/en-us/dynamics365/fin-ops-core/dev-itpro/migration-upgrade/upgrade-home-page>

6. RSAT - <https://docs.microsoft.com/en-us/dynamics365/fin-ops-core/dev-itpro/perf-test/rsat/rsat-overview>

9. References

7. Azure DevOps Services user guide

<https://docs.microsoft.com/de-de/azure/devops/user-guide/index?view=vsts>

8. Development and administration for Finance and Operations

<https://docs.microsoft.com/en-us/dynamics365/unified-operations/dev-itpro/>

9. Branches, build definitions, and environments

<https://docs.microsoft.com/en-us/dynamics365/unified-operations/retail/dev-itpro/new-environments-visual-studio-teams-branch-retail-projects#branches-build-definitions-and-environments>

